

Logistic Regression from Scratch

What is Gradient Descent for Logistic Regression?

Gradient Descent is the standard first-order optimization algorithm. It starts at $w = 0$ and repeatedly nudges the weights in the direction that reduces the binary cross-entropy loss.

Loss Function: Binary Cross-Entropy

$$J(w) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

where $\hat{y} = \sigma(Xw)$ is the sigmoid of the linear score. Unlike MSE, this loss is **convex** in w , so gradient descent is guaranteed to reach the global minimum.

The Gradient

A remarkable result: even though we squash through a non-linear sigmoid, the gradient has the *same form* as linear regression:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) \cdot x_j^{(i)}$$

The only difference: $\hat{y}^{(i)} = \sigma(X^{(i)}w)$ instead of $X^{(i)}w$. In words: *the average of (prediction error $\times$ feature value) across all samples*.

The Update Rule

$$w_j \leftarrow w_j - \alpha \cdot \frac{\partial J}{\partial w_j}$$

All weights are updated simultaneously after computing the full-batch gradient.

Key Hyperparameters

Hyperparameter	Too small	Too large
α (learning rate)	Converges very slowly	Diverges / overshoots
iterations	Underfits	Wastes compute (but won't hurt if α is right)

3 Ways to Implement It

Below we implement gradient descent three times, progressively eliminating explicit Python loops using NumPy vectorization:

```
In [1]: """
Write a Python class that performs logistic regression using gradient descent. The class should take NumPy arrays X
(features with a column of ones for the intercept)
and y (binary target) as input, along with the learning rate alpha and the number of iterations. Return the learned
coefficients (weights) as a NumPy array.

Requirements
Minimize the Binary Cross-Entropy loss function:


$$J(w) = -(1/m) * \sum [ y \log \hat{y} + (1 - y) \log (1 - \hat{y}) ]$$
   where  $\hat{y} = \text{sigmoid}(Xw)$ 

Initialize all weights to zero

Use batch gradient descent (use all samples in each iteration)

The input matrix X has shape (m, n) where m is the number of training examples and n is the number of features
(including the bias column of ones).
The target vector y has shape (m,) with values in {0, 1}.
"""
```

```
Out[1]: '\nWrite a Python class that performs logistic regression using gradient descent. The class should take NumPy arrays X
(features with a column of ones for the intercept) \nand y (binary target) as input, along with the learning rate alpha
and the number of iterations. Return the learned coefficients (weights) as a NumPy array.\n\nRequirements\nMinimize the
Binary Cross-Entropy loss function:\n\n
$$J(w) = -(1/m) * \sum [ y \log \hat{y} + (1 - y) \log (1 - \hat{y}) ]$$
   where  $\hat{y} =$ 
sigmoid(Xw)\n\nInitialize all weights to zero\n\nUse batch gradient descent (use all samples in each iteration)\n\nThe
input matrix X has shape (m, n) where m is the number of training examples and n is the number of features (including
the bias column of ones). \nThe target vector y has shape (m,) with values in {0, 1}.\n'
```

Version	Loops removed	Core idea
3-Loop	None	Fully explicit: iterate over epochs → features → samples
2-Loop	Sample loop	Use <code>np.sum</code> to aggregate over all m samples at once
1-Loop	Sample + feature loops	One matrix multiply updates all w_j simultaneously

Version 1 — 3-Loop (Most Explicit)

```
In [2]: import numpy as np

def sigmoid(z: np.ndarray) -> np.ndarray:
    return 1.0 / (1.0 + np.exp(-z))

import numpy as np

# using 3 loop implementation
class LogisticRegressionGD3Loop:
    def __init__(self, alpha: float = 0.01, iterations: int = 1000):
        self.alpha = alpha
        self.iterations = iterations
        self.theta = None

    def fit(self, X: np.ndarray, y: np.ndarray):
        m, n = X.shape
        y = y.reshape(-1, 1)
        self.theta = np.zeros((n, 1))

        for _ in range(self.iterations): # for each epoch/ iteration
            for j in range(n): # update the parameters for each feature
                gradient_sum = 0
                for i in range(m): # compute the gradient for each training example (loss function)
                    y_hat_i = sigmoid(X[i] @ self.theta)
                    gradient_sum += (y_hat_i - y[i]) * X[i, j]
                gradient = (1 / m) * gradient_sum # cost function gradient
                self.theta[j] = self.theta[j] - self.alpha * gradient
        return self
```

```
def predict_proba(self, X: np.ndarray) -> np.ndarray:
    return sigmoid(X @ self.theta).flatten()
```

3-Loop Walkthrough

```
for each epoch:                                # Loop 1 – repeat until convergence
    for j in range(n):                          # Loop 2 – one weight  $w_j$  at a time
        grad_sum = 0
        for i in range(m):                    # Loop 3 – accumulate over each sample
             $\hat{y}_i = \sigma(X_i \cdot w)$         # sigmoid prediction for sample i
            grad_sum += ( $\hat{y}_i - y_i$ ) *  $X_{ij}$ 
         $w_j -= \alpha * (grad\_sum / m)$ 
```

- **Most readable** — mirrors the math formula exactly, term by term
- **Slowest** — three nested Python loops: $O(\text{iterations} \times n \times m)$ overhead
- `sigmoid(X[i] @ w)` computes $\hat{y}_i$ for the i-th sample
- Each weight's gradient is accumulated sample-by-sample before the update

```

In [3]: # Test Case 1
model = LogisticRegressionGD3Loop(alpha=0.1, iterations=2000)
model.fit(np.array([[1, 0], [1, 1], [1, 2], [1, 3], [1, 4], [1, 5]]),
          np.array([0, 0, 1, 0, 1, 1]))
print("Coefficients:", np.round(model.theta.flatten(), 4))
print("Probabilities:", np.round(model.predict_proba(np.array([[1, 0], [1, 1], [1, 2], [1, 3], [1, 4], [1, 5]])), 4))
# Expected Coefficients: [-3.0209  1.2094]

# Test Case 2
model = LogisticRegressionGD3Loop(alpha=0.1, iterations=2000)
model.fit(
    np.array([[1, 1, 1], [1, 1, 2], [1, 2, 1], [1, 2, 2], [1, 2, 3],
              [1, 3, 2], [1, 3, 3], [1, 4, 3], [1, 4, 4], [1, 3, 1]]),
    np.array([0, 0, 0, 1, 0, 0, 1, 1, 1, 1]))
)
print("Coefficients:", np.round(model.theta.flatten(), 4))
# Expected Coefficients: [-4.8562  1.8796  0.0951]

```

Coefficients: [-3.0209 1.2094]

Probabilities: [0.0465 0.1405 0.3539 0.6473 0.8602 0.9537]

Coefficients: [-4.8562 1.8796 0.0951]

Version 2 — 2-Loop (Vectorize Over Samples)

```
In [4]: # using 2 loop implementation
class LogisticRegressionGD2Loop:
    def __init__(self, alpha: float = 0.01, iterations: int = 1000):
        self.alpha = alpha
        self.iterations = iterations
        self.theta = None

    def fit(self, X: np.ndarray, y: np.ndarray):
        m, n = X.shape
        y = y.reshape(-1, 1)
        self.theta = np.zeros((n, 1))

        for _ in range(self.iterations): # for each epoch/ iteration
            for j in range(n): # update the parameters for each feature
                y_hat = sigmoid(X @ self.theta)
                gradient = (1 / m) * np.sum((y_hat - y) * X[:, j].reshape(-1, 1)) # cost function gradient
                self.theta[j] = self.theta[j] - self.alpha * gradient
            return self

    def predict_proba(self, X: np.ndarray) -> np.ndarray:
        return sigmoid(X @ self.theta).flatten()
```

2-Loop Walkthrough

```
for each epoch:                # Loop 1 – still iterating over epochs
    for j in range(n):          # Loop 2 – still one weight at a time
         $\hat{y}$  =  $\sigma(X @ w)$       # shape (m, 1) – all m predictions at once
        grad = (1/m) * np.sum(( $\hat{y}$  - y) * X[:, j])
         $w_j$  -=  $\alpha * \text{grad}$ 
```

- **Sample loop eliminated:** `np.sum(...)` replaces the explicit sample-wise accumulation
- `sigmoid(X @ w)` computes all m predictions $\hat{y}$ in one matrix multiply
- `X[:, j]` selects the j-th feature column across all m samples
- Still loops over features — partially vectorized

```

In [5]: # Test Case 1
model = LogisticRegressionGD2Loop(alpha=0.1, iterations=2000)
model.fit(np.array([[1, 0], [1, 1], [1, 2], [1, 3], [1, 4], [1, 5]]),
          np.array([0, 0, 1, 0, 1, 1]))
print("Coefficients:", np.round(model.theta.flatten(), 4))
print("Probabilities:", np.round(model.predict_proba(np.array([[1, 0], [1, 1], [1, 2], [1, 3], [1, 4], [1, 5]])), 4))
# Expected Coefficients: [-3.0209  1.2094]

# Test Case 2
model = LogisticRegressionGD2Loop(alpha=0.1, iterations=2000)
model.fit(
    np.array([[1, 1, 1], [1, 1, 2], [1, 2, 1], [1, 2, 2], [1, 2, 3],
              [1, 3, 2], [1, 3, 3], [1, 4, 3], [1, 4, 4], [1, 3, 1]]),
    np.array([0, 0, 0, 1, 0, 0, 1, 1, 1, 1]))
)
print("Coefficients:", np.round(model.theta.flatten(), 4))
# Expected Coefficients: [-4.8562  1.8796  0.0951]

```

Coefficients: [-3.0209 1.2094]

Probabilities: [0.0465 0.1405 0.3539 0.6473 0.8602 0.9537]

Coefficients: [-4.8562 1.8796 0.0951]

Version 3 — 1-Loop (Fully Vectorized)

```
In [6]: # using 1 loop (fully vectorized) implementation
class LogisticRegressionGD:
    def __init__(self, alpha: float = 0.01, iterations: int = 1000):
        self.alpha = alpha
        self.iterations = iterations
        self.theta = None

    def fit(self, X: np.ndarray, y: np.ndarray):
        m, n = X.shape
        y = y.reshape(-1, 1)
        self.theta = np.zeros((n, 1))

        for _ in range(self.iterations): # for each epoch/ iteration
            y_hat = sigmoid(X @ self.theta)
            gradient = (1 / m) * X.T @ (y_hat - y) # cost function gradient
            self.theta = self.theta - self.alpha * gradient
        return self

    def predict_proba(self, X: np.ndarray) -> np.ndarray:
        return sigmoid(X @ self.theta).flatten()
```

1-Loop (Fully Vectorized) Walkthrough

for each epoch:	# The only loop
$\hat{y} = \sigma(X @ w)$	# shape (m, 1) — all predictions
$\text{grad} = (1/m) * X^T @ (\hat{y} - y)$	# shape (n, 1) — gradient for ALL w at once
$w -= \alpha * \text{grad}$	# update ALL weights simultaneously

- `sigmoid(X @ w)` — shape (m, 1): $\hat{y}$ (probabilities) for every sample in one op
 - `$\hat{y} - y$` — shape (m, 1): prediction errors for all samples
 - `$X.T @ (\hat{y} - y)$` — shape (n, 1): computes $\sum_i (\hat{y}^{(i)} - y^{(i)}) \cdot x_j^{(i)}$ for **all j at once** — both the sample loop and feature loop are gone
 - **Fastest in practice** — NumPy delegates to optimized BLAS/LAPACK routines
-

Performance Summary

Version	Python Loops	Dominant cost	Relative speed
3-Loop	iterations × n × m	Interpreted Python	Slowest
2-Loop	iterations × n	Partial NumPy ops	Medium
1-Loop	iterations	Pure NumPy (BLAS)	Fastest

Takeaway: Always prefer the fully vectorized form. The 1-loop version is what you would use in production code.

```
In [7]: # Test Case 1
model = LogisticRegressionGD(alpha=0.1, iterations=2000)
model.fit(np.array([[1, 0], [1, 1], [1, 2], [1, 3], [1, 4], [1, 5]]),
          np.array([0, 0, 1, 0, 1, 1]))
print("Coefficients:", np.round(model.theta.flatten(), 4))
print("Probabilities:", np.round(model.predict_proba(np.array([[1, 0], [1, 1], [1, 2], [1, 3], [1, 4], [1, 5]])), 4))
# Expected Coefficients: [-3.0202  1.2091]

# Test Case 2
model = LogisticRegressionGD(alpha=0.1, iterations=2000)
model.fit(
    np.array([[1, 1, 1], [1, 1, 2], [1, 2, 1], [1, 2, 2], [1, 2, 3],
              [1, 3, 2], [1, 3, 3], [1, 4, 3], [1, 4, 4], [1, 3, 1]]),
    np.array([0, 0, 0, 1, 0, 0, 1, 1, 1, 1]))
print("Coefficients:", np.round(model.theta.flatten(), 4))
# Expected Coefficients: [-4.8441  1.8751  0.0949]
```

Coefficients: [-3.0202 1.2091]

Probabilities: [0.0465 0.1405 0.3539 0.6473 0.8601 0.9537]

Coefficients: [-4.8441 1.8751 0.0949]